Язык программирования C++

Наследование

Доцент кафедры ИВТ, к.т.н.
Проскурин Александр Викторович

Наследование

Наследование – это отношение, связывающее классы, один из которых является **базовым** (**родительским**), а другой создается на его основе (**производный**, **наследник**).

Наследование заключается в том, что класс-наследник приобретает поля и методы родительского класса и добавляет к ним собственные.

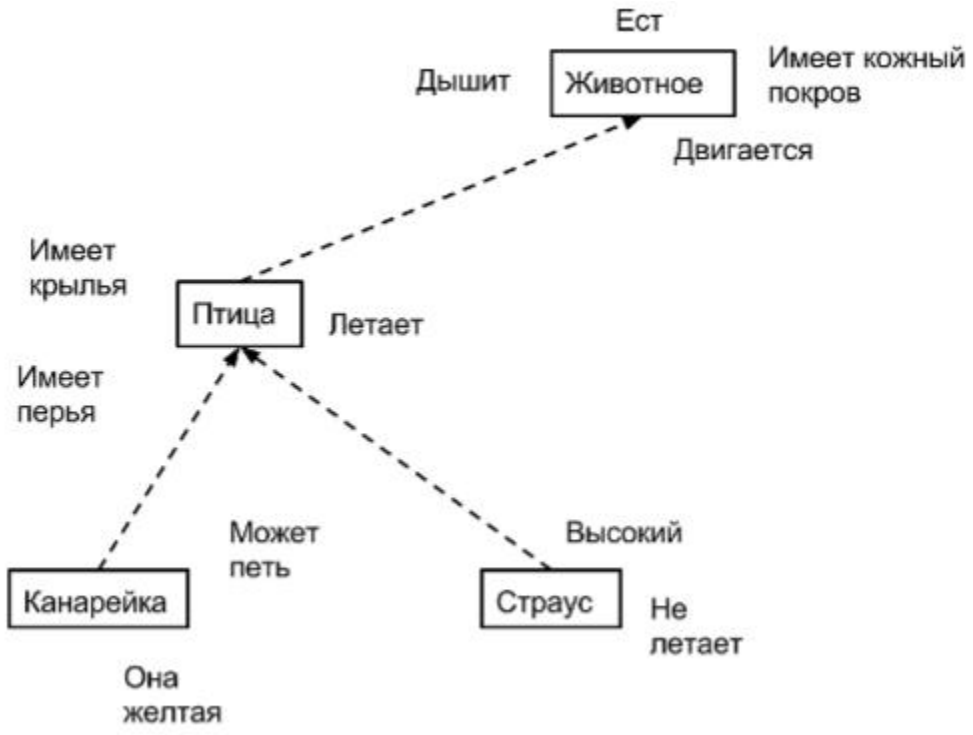
Таким образом, возможно создание **иерархии классов**, в которой базовые классы обладают более общими элементами, а производные классы их конкретизируют (расширяют).

В свою очередь производный класс может служить базовым для другого класса.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Преимущества наследования

Принципы наследования неявно применяются при запоминании информации об окружающем мире.



Так, вместо запоминания всех свойств каждой отдельной птицы, мы запоминаем только её отличительные черты (для канарейки это умение петь и жёлтый цвет), перенося общие свойства (такие как умение летать и наличие крыльев) на более абстрактное понятие «птица».

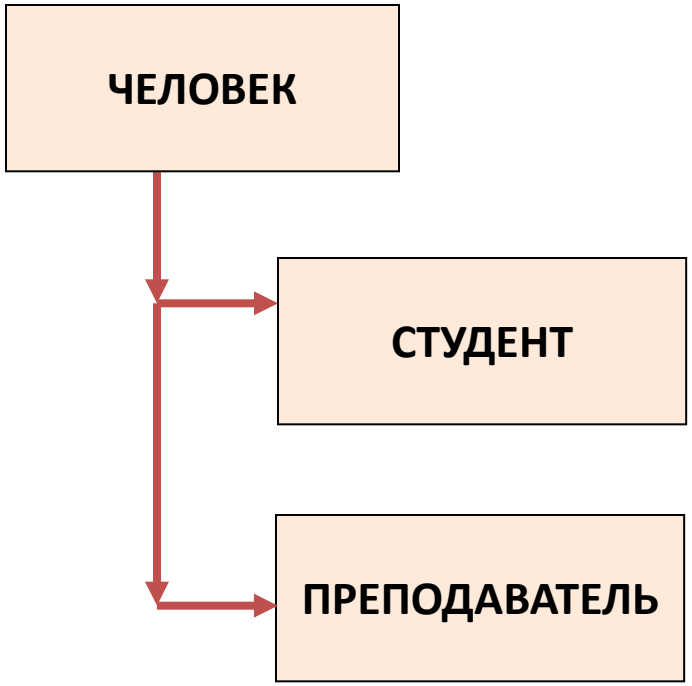
Эксперименты показали, что ответ на вопрос «Может ли канарейка петь?» требует меньше времени, чем на вопрос «Может ли канарейка дышать?»

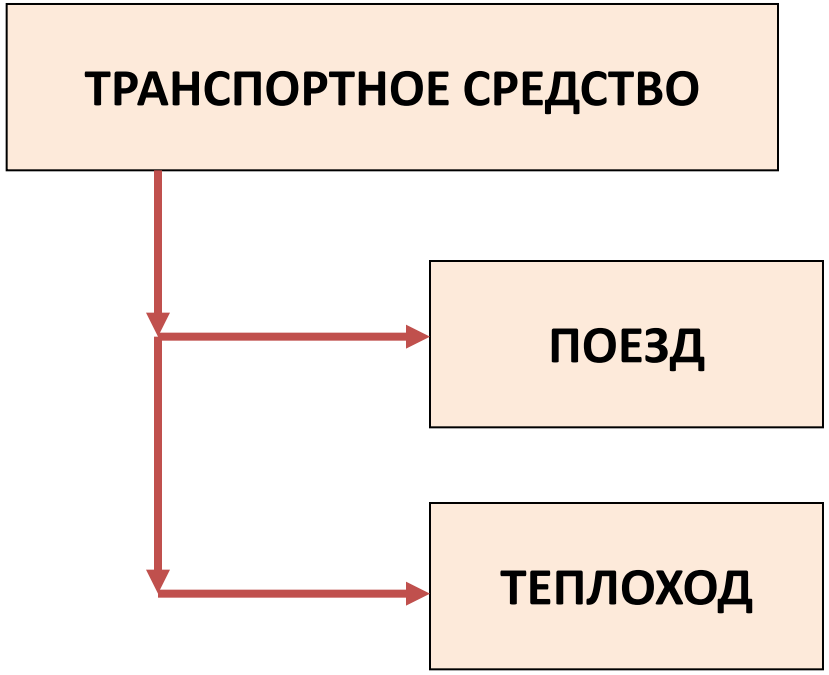
© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Преимущества наследования

Наследование позволяет:

- исключить из программы повторяющиеся фрагменты кода;
- упростить модификацию программы;
- использовать библиотеки классов для того, чтобы взять за основу классы, разработанные кем-то другим, и создать наследников с требуемыми свойствами





© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Объявление производного класса

Основная форма синтаксиса объявления производного класса следующая:

class имя_производного_класса :

< режим_доступа > имя_базового_ класса

{

//..... <определение компонентов>

};

```
class A
{
    //…
};
class X: public A
{
    //…
};
Здесь A – базовый класс,
      X – производный класс
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Режимы доступа при наследовании

Режим доступа	Спецификатор доступа в базовом классе	Доступ в производном классе
private (по умолчанию)	private	Элементы базового класса не доступны
	protected	private
	public	private
protected	private	Элементы базового класса не доступны
	protected	protected
	public	protected
public	private	Элементы базового класса не доступны
	protected	protected
	public	public

Private-элементы базового класса остаются для него собственными и не доступны в производном классе.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Фрагмент примера наследования

```
class Human { // базовый класс
  string name;
  string last_name;

public:
  string get_full_name() const
  {
    string res = name + " "
                + last_name;
    return res;
  }
  // конструкторы и т.д.
};
```

```
class Student: public Human {
  int* scores;
  int num;
public:
  float get_avg_score() const
  {
    float avg = 0;
    for(int i=0; i<num; i++)
      avg += scores[i];
    return avg/num;
  }
  ~Student() {
    delete [] scores;
  } // конструкторы и т.д.
};
```

```
int main() { //Здесь нужен будет
 Student st; // констр. с парам.
 cout<< st.get_full_name()<< " "
     << st.get_avg_score();
}
```

```
Student::| Human::name // string
         |        last_name // string
         | scores // int*
         | num // int
```

Доступ к наследуемым элементам

Если в класс Student попробовать добавить метод print():

```
void Student::print() const
{
  // нет доступа к закрытым полям базового класса
  cout << "Студент " << last_name << " " << name; // Ошибка!

  // мы можем вызывать открытые методы базового класса
  cout << "Студент " << get_full_name(); // ok
  // а также любые методы своего собственного класса
  cout << " имеет средний балл " << get_avg_score();
}
```

Важно! Закрытый элемент класса доступен **только** для других элементов **этого класса**.

Для прямого обращения к **элементам базового** класса необходимо объявить их со спецификатором **доступа protected** или использовать **общедоступные методы** базового класса.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример прямого доступа к наследуемым элементам

```
class Human { // базовый
protected:
  string name;
  string last_name;
public:

  // конструкторы и т.д.
};
```

```
int main() { //Здесь нужен
 Student st; // будет
 st.print(); // констр. с
 return 0;   // параметр.
}
```

Теперь мы можем обращаться к полям базового класса напрямую.

```
class Student: public Human {
  int* scores;
  int num;
public:
  float get_avg_score() const {
    float avg = 0;
    for(int i=0; i<num; i++)
      avg += scores[i];
    return avg/num;
  }
  void print() const
  {
    cout << "Студент " << name
         << " " << last_name; //ok
    cout << " со средним баллом "
         << get_avg_score();
  }
  // конструкторы и т.д.
};
```

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример обращения к наследуемым элементам при совпадении имён

```
class Human { // базовый
protected:
  string name;
  string last_name;
public:
  void print() const
  {
    cout << "Человек "
         << name << " "
         << last_name;
  }
  // конструкторы и т.д.
};
```

```
class Student: public Human {
  int* scores;
  int num;
public:
  float get_avg_score() const {
    float avg = 0;
    for(int i=0; i<num; i++)
      avg += scores[i];
    return avg/num;
  }
  void print() const
  {
    Human::print();
    cout << " со средним баллом "
         << get_avg_score();
  }
  // конструкторы и т.д.
};
```

Для того, чтобы обратиться именно к элементу базового класса, необходимо указать имя этого класса и вставить операцию доступа к вложенным именам **::**

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Правила наследования методов

Производный класс наследует protected- и public- методы своих базовых классов.

Однако **конструкторы**, **деструкторы** и **operator= не наследуются**, но могут быть **вызваны**.

1) **Конструкторы**. Порядок вызова определяется правилами:

- Если в конструкторе производного класса явный вызов базового класса отсутствует, то **автоматически вызывается конструктор по умолчанию** (без параметров).

```
class Basov
{
protected:
  int x;
public:
  Basov() {
    x = 0;
  }
};
```

```
class Proisv : public Basov
{
  int y;
public:
  // Будет автоматически вызван
  // конструктор Basov() и
  // полю x присвоится 0
  Proisv(int j) : y(j) {}
};
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Правила наследования методов (продолжение для конструкторов)

- Если в конструктор базового класса необходимо передать параметры, то он **должен быть явным образом вызван в списке инициализации** конструктора производного класса:

```
class Basov
{
protected:
  int x;
public:
  Basov(int i) : x(i) {}
};
```

```
class Proisv : public Basov
{
  int y;
public:
  Proisv(int i, int j) : Basov(i)
                       , y(j) {}
};
```

- В иерархии, состоящей из нескольких уровней, **конструкторы вызываются начиная с самого верхнего уровня** (самого базового класса).
- В случае наличия нескольких базовых классов, их конструкторы вызываются в **порядке объявления в заголовке класса-наследника**.
- Если в классе X есть поля, являющиеся объектами, то их конструкторы вызываются в порядке объявления до вызова конструктора класса X.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Правила наследования методов (продолжение для присваивания)

2) Перегрузка операции присваивания (явное копирование). Компилятор создаёт эту перегрузку автоматически (копируются значения всех полей). Если есть указатель, то нужно перегружать самим, обязательно сделав проверку на самоприсваивание.

```
class Basov {
protected:
 char* x;
public:
 Basov& operator=(const Basov &s)
 {
  if(this != &s) {
   delete [] x;
   x = new char[strlen(s.x) + 1];
   strcpy(x, s.x);
  }
  return *this;
 }
};
```

```
class Proisv : public Basov
{
 int y;
public:
 Proisv& operator=(const Proisv
                              &s)
 {
  if(this != &s) {
    Basov::operator=(s);
    y = s.y;
  }
  return *this;
 }
};
```

Обратите внимание, что вместо ссылки на объект базового класса передаётся ссылка на объект производного.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Правила наследования методов (продолжение для присваивания)

2) Перегрузка операции присваивания (передача копии). Компилятор также создаёт перегрузку автоматически, но при наличие поля-указателя нужно перегружать самим через передачу копии и обмен значений полей.

```
class Basov
{
protected:
 char* x;
public:
 Basov& operator=(Basov s)
 {
   std::swap(x, s.x);
   return *this;
 }
};
```

Передаём по значению – в этом месте вызывается конструктор копирования и если он не реализован правильно (с копированием динамической памяти), то будут ошибки после срабатывания деструктора.

```
class Proisv : public Basov
{
 int y;
public:
 Proisv& operator=(Proisv s)
 {
   Basov::operator=(s);
   std::swap(y, s.y);
   return *this;
 }
};
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Правила наследования методов (продолжение для деструктора)

3) **Деструктор**. Правила:

- Если в производном классе не описан явным образом деструктор, то он создаётся автоматически и вызывает деструктор базового класса.
- **Деструктор базового класса не следует вызывать явным образом**, это будет сделано автоматически.
- В иерархии, состоящей из нескольких уровней, **деструкторы вызываются в порядке, обратном вызову конструкторов** (начиная с деструктора производного класса).

```
class Basov {
protected:
  char *st;
public:
  ~Basov() {
    delete [] st;
  }
};
```

```
class Proisv : public Basov
{
  char *des;
public:
  ~Proisv() {
    delete [] des;
  }
};
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

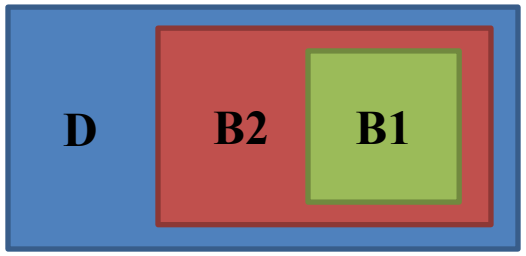
Демонстрация порядка вызовов конструкторов и деструкторов

```
class B1 {
public:
  B1() { cout << "Раб. конструктора B1\n"; }
  ~B1() { cout << "Раб. деструктора B1\n"; }
};

class B2 : public B1 {
public:
  B2() { cout << "Раб. конструктора B2\n"; }
  ~B2() { cout << "Раб. деструктора B2\n"; }
};

class D : public B2 {
public:
  D() { cout << "Раб. конструктора D\n"; }
  ~D() { cout << "Раб. деструктора D\n"; }
};
```

```
int main() {
   D* ob = new D;
   delete ob;
   return 0;
}
```

Результаты работы программы:

```
Раб. конструктора B1
Раб. конструктора B2
Раб. конструктора D
Раб. деструктора D
Раб. деструктора B2
Раб. деструктора B1
```



© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Указатели на производные классы

В общем случае указатель одного типа не может указывать на объект другого типа. Однако в C++ **указатель на базовый класс может указывать на объект производного класса**, полученного из этого базового класса.

Предположим, что имеется базовый класс **Basov** и его производный класс **Proisv**. В C++ любой указатель типа **Basov*** может также указывать на объект типа **Proisv**. Например, если имеются объявления переменных:

```
Basov *p;   // указатель на объект типа Basov
Basov  B_obj; // объект типа Basov
Proisv P_obj; // объект типа Proisv
```

то следующие присваивания абсолютно законны:

```
p = &B_obj; // p указывает на объект типа Basov
p = &P_obj; /* p указывает на объект типа Proisv,
           являющийся объектом, порожденным от Basov */
```

Однако **без использования виртуальных методов** при попытке вызова `p->любой_метод()` всегда будут вызываться методы базового класса (т.е. соответствующие **типу указателя, а не фактическому типу объекта**).

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Виртуальные методы

К механизму **виртуальных методов** (virtual methods) обращаются в тех случаях, когда в базовый класс необходимо поместить метод, который должен по-разному выполняться в производных классах.

Виртуальные методы позволяют производным классам определить **разные версии методов базового класса** и реализуют идею «один интерфейс – множество методов», лежащую в основе полиморфизма.

Виртуальные методы используются для поддержки **динамического полиморфизма**, работающего во время выполнения программы.

Класс, содержащий один или несколько виртуальных методов, называется **полиморфным классом**.

Объект полиморфного класса называют **полиморфным** объектом.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Синтаксис виртуального метода

Виртуальный метод объявляется в базовом классе с использованием ключевого слова **virtual**:

```
virtual тип_возвр_зн имя_метода (список_параметров);
```

Когда этот метод переопределяется в производном классе, то в новом стандарте прописывается ключевое слово **override** уже после объявления метода:

```
тип_возвр_зн имя_метода (список_параметров) override;
```

```
class Human {
protected:
  string name;
  string last_name;
public:
  virtual void print() const
  {
    cout << "Человек "
         << name << " "
         << last_name;
  }
  // конструкторы и т.д.
};
```

```
class Student: public Human {
  int* scores;
  int num;
public:
  void print() const override {
    cout << "Студент ";
    Human::print();
  }
  // конструкторы и т.д.
};
```

```
int main() {
  Human *h = new Student();
  h->print();
  delete h;
}
```

Вызовется Student::print()

Вызовется Human::~Human()
Деструктор должен быть виртуальным

Указатели на классы и виртуальные методы

```
class Base {
public:
 // определение виртуального метода
 virtual void who() {
  cout << "Базовый класс\n";
 }
};
class First : public Base {
public:
 // определение who() для First
 // реализация отличается от Base
 void who() override {
  cout << "Класс First\n";
 }
};
class Second : public Base {
public:
 // определение who() для Second
 void who() override {
  cout << "Класс Second\n";
 }
};
```

```
int main() {
 Base base, *p;
 First first;
 Second second;
 p = &base;
 // доступ к Base::who
 p->who();
 p = &first;
 // доступ к First::who
 p->who();
 p = &second;
 // доступ к Second::who
 p->who();
 return 0;
}
```

Результаты работы программы:

```
Базовый класс
Класс First
Класс Second
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Указатели на классы и виртуальные методы

```
class Base {
public:
 // объявление виртуального метода
 virtual void Method()
 { cout << "Базовый\n"; }
};
class Nasled : public Base {
public:
 // virtual здесь допустимо,
 // но избыточно
 void Method() override
 { cout << "Производный\n"; }
};
// определение внешней функции
void fn(Base *basePtr)
{// вызов метода через указатель
 basePtr->Method(); // на базовый класс
}
```

```
int main() {
 Base base;
 Nasled nas;
 cout << "Базовый - ";
 // вызов из функции fn
 // виртуального метода
 // базового класса
 fn(&base);
 cout << "Производный - ";
 // вызов из функции fn
 // виртуального метода
 // производного класса
 fn(&nas);
 return 0;
}
```

Результаты работы программы:

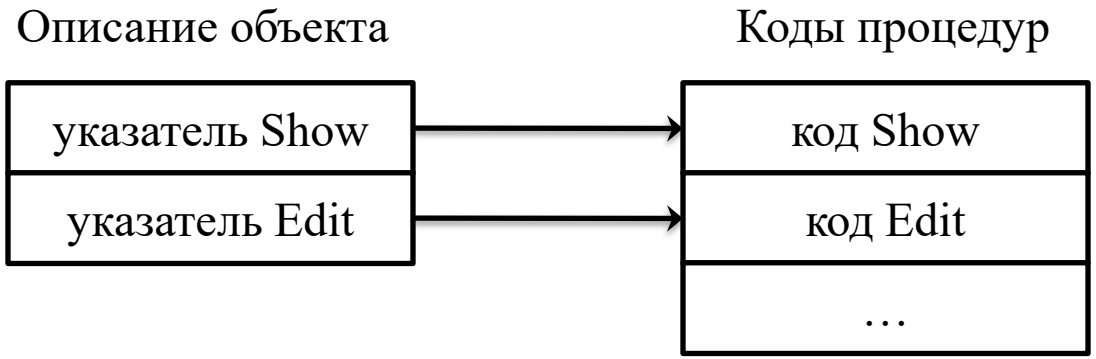
```
Базовый - Базовый
Производный - Производный
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Раннее и позднее связывание

Необходимо **связывать вызов метода с его реализацией**:

Раннее (статическое) связывание – т.е. связывание **на этапе компиляции**. Каждый вызов метода класса замещается на указатель с конкретным адресом памяти, где находится код этого метода.



Позднее (динамическое) связывание – связь метода с реализующей процедурой определяется **на этапе выполнения программы**. На этапе компиляции ещё неизвестен конкретный тип объекта (базового класса или какого-то наследника).

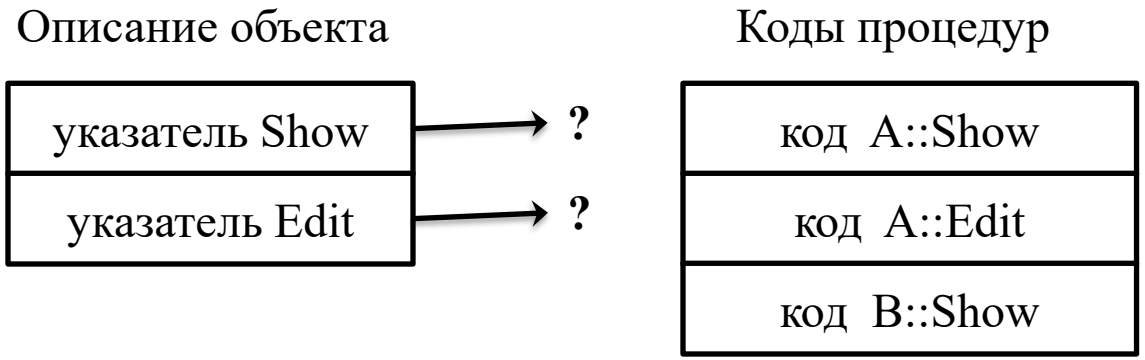


Таблица виртуальных методов

Для виртуальных методов компилятор формирует **таблицу виртуальных методов** (Virtual Method Table, VMT).

Для каждого класса создается одна таблица.

- **Таблицу виртуальных методов можно представить как обычную таблицу из двух столбцов.**
- **В каждой строке: заголовок виртуального метода и адрес его реализации в конкретном классе.**
- В пределах одной иерархии позиции виртуальных методов неизменны.

Каждый объект во время выполнения программы должен иметь доступ к VMT.

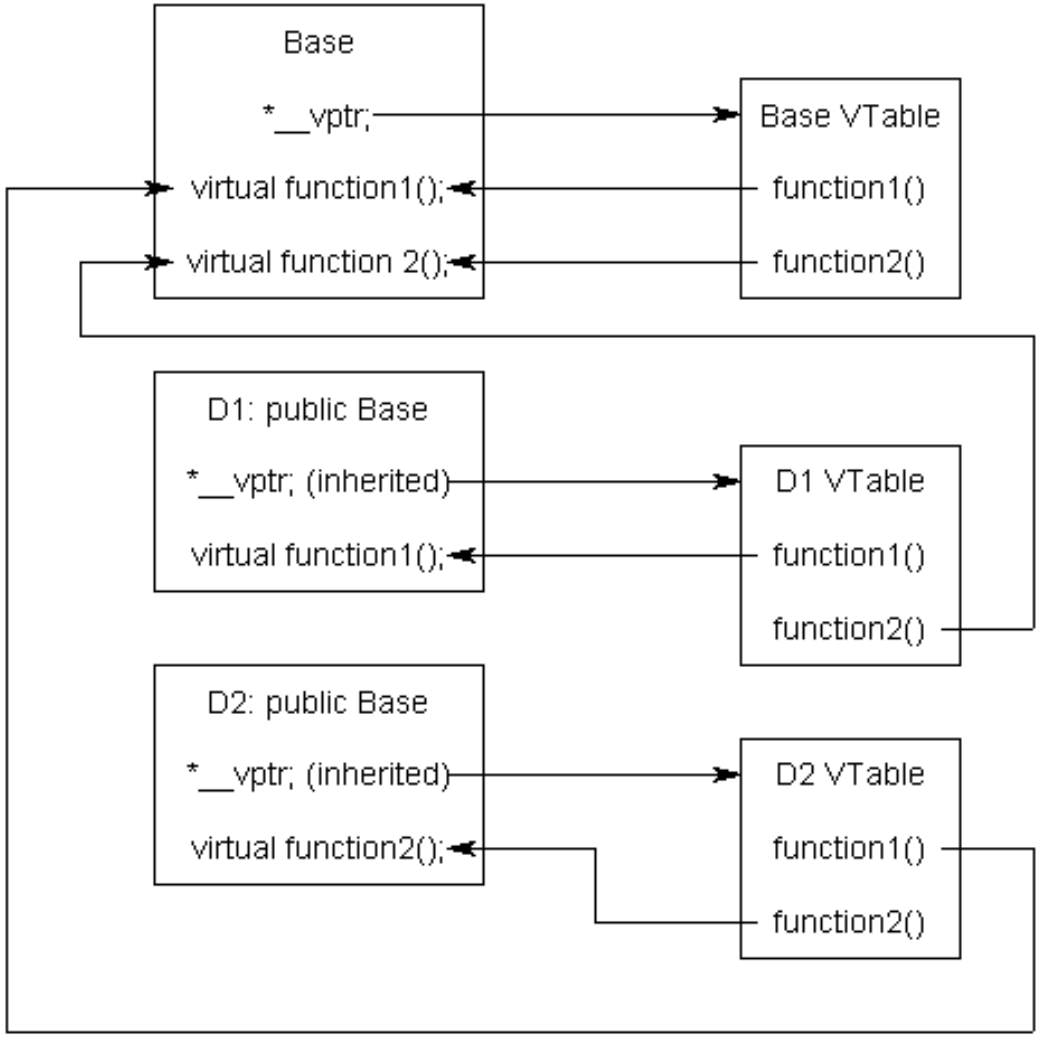
Связь объекта с VMT устанавливается с помощью специального кода, автоматически помещаемого компилятором в конструктор класса.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Таблица виртуальных методов

Таблица виртуальных методов размещается в инициализированной части сегмента данных. Наглядно можно представить следующим образом:

```
class Base {
public:
 virtual void function1();
 virtual void function2();
 // другие не вирт. методы
};


class D1 : public Base {
public:
void function1() override;
 // другие не вирт. методы
};


class D2 : public Base {
public:
void function2() override;
 // другие не вирт. методы
};
```



© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Особенности работы с виртуальными методами

- Если в базовом классе метод объявлен как виртуальный, то метод, объявленный в производном классе **с тем же именем, типом возвращаемого значения и списком параметров**, автоматически становится виртуальным. Если хоть что-то отличается, то механизм виртуальных методов игнорируется.

- **Виртуальные методы наследуются**. Переопределять их в производном классе требуется только при необходимости **задать отличающиеся действия**.

- Если виртуальный метод переопределен, то вызвать метод базового класса можно через операцию доступа к области видимости (**имя_базового_класса::имя_виртуального_метода**).

- Виртуальный метод **не может** объявляться с модификатором **static**.

- В классе может быть определен **чистый виртуальный метод** (не имеющий тела):

```
virtual тип_возвр_зн имя_метода (список_параметров)=0;
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример переопределения виртуальных методов

```
class Basov {
public:
  virtual void vf1();
  virtual void vf2();
  virtual void vf3();
  void f();
};

class Proisv : public Basov {
public:
  // виртуальный метод, причем слово virtual избыточно
  virtual void vf1();
  // виртуальный метод
  void vf2(); // override можно не прописывать
  // не виртуальный, т.к. использован другой список параметров
  void vf2(int);
  // не виртуальный, т.к. изменен тип возврата
  char vf3();
  // не виртуальный, т.к. не объявлен в базовом классе как virtual
  void f();
};
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Абстрактные классы

Абстрактный класс – это класс с одним или большим количеством чистых виртуальных методов.

Абстрактные классы предназначены **для представления общих понятий**, которые предполагается конкретизировать в производных классах.

Используется, когда создавать экземпляры базового класса не имеет смысла, потому что никакие реальные объекты им не соответствуют.

Абстрактные классы служат только для порождения потомков. Как правило, в абстрактных классах создается набор методов, которые каждый из потомков будет реализовывать по-своему.

Чистый виртуальный метод – это виртуальный метод, у которого есть объявление (заголовок) и нет реализации (тела метода). Вместо него записывается признак **= 0**:

```
virtual тип_возвр_зн имя_метода (список_параметров) = 0;
```

Правила языка C++ **запрещают создание объектов** абстрактных классов.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример абстрактного класса

```
class Shape { // Абстрактный класс фигуры
public:       // Полей может и не быть
  virtual double area() const = 0; // не обязательно все
  virtual string myName() const = 0; // методы чистые вирт.
  virtual Shape* getCopy() const = 0;
};
class Circle : public Shape {
  double radius;
public:
  double area() const {
    return 3.14*radius*radius;
  }
  string myName() const {
    return "Circle";
  }
  Shape* getCopy() const {
    return new Circle(*this);
  }
};
```

```
class Rectangle: public Shape{
  double height, width;
public:
  double area() const {
    return height * width;
  }
  string myName() const {
    return "Rectangle";
  }
  Shape* getCopy() const {
    return new Rectangle(*this);
  }
};
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример абстрактного класса (продолжение)

```
int main() {
  const int n = 100;
  //массив указателей на баз.
  Shape* p[n];   //класс
  //создаём объекты наслед.
  //и сохраняем в массив
  int type;
  for(int i = 0; i<n; i++) {
    cin >> type;
    if(type == 0)
      p[i] = new Rectangle();
      //заполнение полей
    else if(type == 1)
      p[i] = new Circle();
      //заполнение полей
  }
  Shape* obj = p[0].getCopy();
  cout << obj->myName();
```

```
  // вычисляем суммарную
  // площадь объектов
  double ttl_area = 0;
  for(int i = 0; i<n; i++)
    ttl_area += p[i]->area();
  cout << ttl_area;
  // находим среднюю площадь
  // объектов класса Circle
  double crc_sum = 0;
  int num_crc = 0;
  for(int i = 0; i<n; i++)
   if(p[i]->myName()=="Circle")
   {
      crc_sum += p[i]->area();
      num_crc++;
   }
  cout << crc_sum / num_crc;
  return 0;
}
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Множественное наследование

Множественное наследование – производный класс наследуется от двух и более базовых классов.

Определение производного класса:

```
class имя_производного_класса
                : <режим_доступа_1> имя_базового_класса_1
                , <режим_доступа_2> имя_базового_класса_2
                , ...
{
  //............ <определение компонентов>
};
```

```
class X
{
protected:
 int a;
public:
 X(int i)
       : a(i) {}
};
```

```
class Y
{
protected:
 int b;
public:
 Y(int i)
       : b(i) {}
};
```

```
class Z : public X, public Y
{
public:
 Z(int x, int y)
              : X(x), Y(y) {}
 int mul_ab() { return a*b; }
};
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример множественного наследования

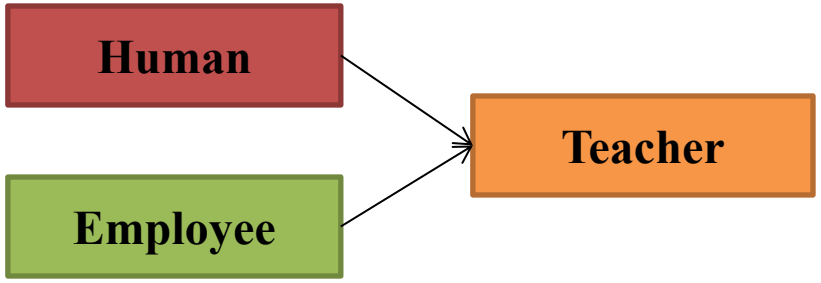
Есть два класса Human (человек) и Employee (работник). Необходимо создать класс Teacher (учитель), который является наследником обоих классов.

```
class Human
{
private:
  string name;
  int age;
public:
  Human(const string &n, int a) : name(n), age(a) { }
  const string & get_name() const { return name; }
  int get_age() const { return age; }
  virtual void print() const { cout << name << " " << age; }
};

class Employee
{
private:
  string post;
  double wage;
public:
  Employee(const string &p, double w) : post(p), wage(w) {}
  const string & get_post() const { return post; }
  double get_wage() const { return wage; }
  virtual void print() const { cout << post <<" " << wage; }
};
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример множественного наследования

```
// Класс Teacher открыто наследует классы Human и Employee
class Teacher: public Human, public Employee
{
private:
  int grade;
public:
  Teacher(const string &_name, int _age,
          const string &_post, double _wage, int _grade)
         : Human(_name, _age)
         , Employee(_post, _wage), grade(_grade) {}
  void print() const {
    Human::print(); // при совпадении имён нужно явно
    Employee::print(); // указать базовый класс
    cout << " " << grade;
  }
};

int main()
{
  Teacher t("Петров П.П.", 55, "Доцент ИУС", 40000, 4);
  t.print();
  return 0;
}
```



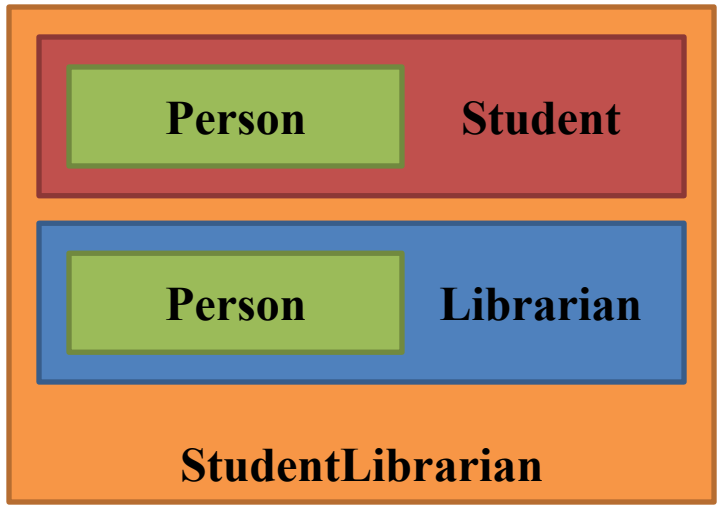

```
Выведет: Петров П.П. 55Доцент ИУС 40000 4
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Проблема ромбовидного наследования

Ромбовидное наследование – возможность многократного включения базового класса.

```
class Person
{
  // ...
};

class Student : public Person
{
  // ...
};

class Librarian : public Person
{
  // ...
};

class StudentLibrarian : public Student, public Librarian
{
  // ...
};
```



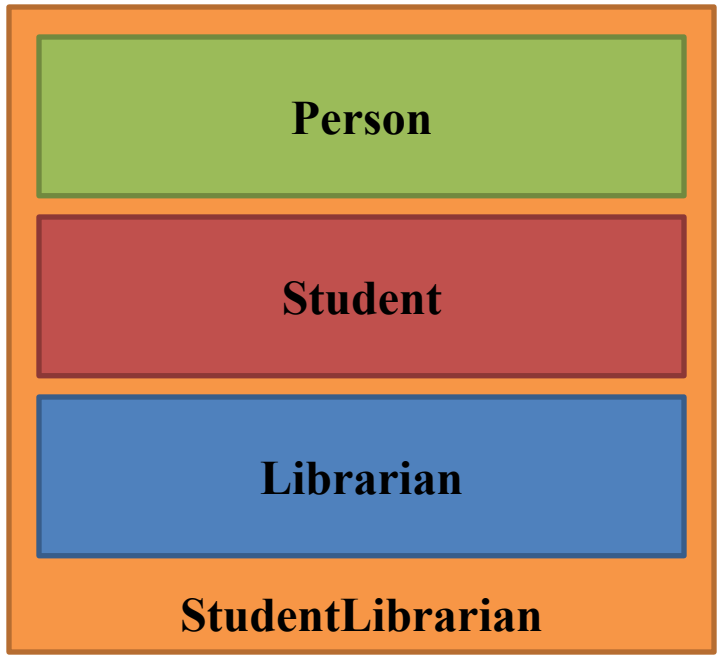
Объект StudentLibrarian будет содержать 2 экземпляра класса Person, унаследованных от обоих родителей

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Решение проблемы ромбовидного наследования

Базовый класс можно объявить виртуальным базовым классом, используя запись:

```
class Person
{
  // ...
};

class Student : virtual Person
{
  // ...
};

class Librarian : virtual Person
{
  // ...
};

class StudentLibrarian : public Student, public Librarian
{
  // ...
};
```



© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Итоги

- Механизм наследования позволяет устанавливать наследственную иерархию между классами. Реализация этого механизма в программе делает ее менее громоздкой и более понятной. Наследование позволяет создавать библиотеки классов и использовать их в программах, что значительно сокращает время написания программ.

- К механизму виртуальных методов необходимо обращаться при организации иерархии классов тогда, когда каждый производный класс требует своего варианта метода, уже определенного в базовом классе. Классы, включающие виртуальные методы, называются полиморфными. Они играют особую роль в ООП, так как обеспечивают большую гибкость программ.

- Виртуальные методы обрабатываются по принципу позднего связывания, когда адрес метода в памяти извлекается из таблицы виртуальных методов в зависимости от конкретного типа объекта, вызвавшего метод.

- Абстрактные классы предназначены для представления общих понятий, которые затем будут конкретизированы в производных классах. Абстрактный класс может быть использован только в качестве базового класса.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева